

Before you turn this problem in, make sure everything runs as expected. First, **restart the kernel** (in the menu bar, select Kernel → Restart) and then **run all cells** (in the menu bar, select Cell → Run All).

Below, please fill in your name and collaborators:

NAME = "Darrah Borinaga"

Assignment 2 - Time Series Analysis

(30 points total)

Assignment tasks:

In this assignment you will conduct time series analysis of the financial data.

1. Setup your environment to access and download latest stock data. You will use the Yahoo Finance library. To install it, you will need to run this code: `pip install yfinance`
2. (4 points) Download the **adjusted** close prices for FB, MMM, IBM and AMZN for the last 60 months.
3. (6 points) Resample the data to get prices for the end of the **business** month. Select the **Adjusted Close** for each stock.
4. (6 points) Use the pandas `autocorrelation_plot()` function to plot the autocorrelation of the adjusted month-end close prices for each of the stocks.
 - Are they autocorrelated?
 - Provide short explanation.
5. (8 points)
 - Calculate the monthly returns for each stock using the "shift trick" explained in the lecture, using `shift()` function.
 - Use pandas `autotocorrelation_plot()` to plot the autocorrelation of the monthly returns.
 - Are the returns autocorrelated? Provide short explanation.
6. (6 points)
 - Combine all 4 time series (returns) into a single DataFrame,
 - Visualize the correlation between the returns of all pairs of stocks using a scatter plot matrix (use `scatter_matrix()` function from `pandas.plotting`).
 - Explain the results. Is there any correlation?

NOTES:

1. In this assignment, please make sure the DataFrame(s) do not contain any NAs before you plot autocorrelations or scatter matrix.
 2. Both options explained below use `pandas-datareader` package for remote data access. To install it, type the following in a command window: `conda install pandas-datareader`. You will also need to install one or more of the following packages `yfinance`.
-

Import all the relevant libraries and follow the examples below for importing data.

```
import yfinance as yf
from pandas_datareader import data as pdr
import pandas as pd
pd.core.common.is_list_like = pd.api.types.is_list_like
import datetime
import pandas_datareader.data as web

import datetime

start = datetime.datetime.today() - datetime.timedelta(days=365*5)

# Option 1) Using pandas datareader and Yahoo Finance
#yf.pdr_override()

#amzn = pdr.get_data_yahoo('AMZN', start= start )

# Option 2) Using yfinance library only to download AMZN data

amzn = yf.download(
    'AMZN',
    start=start,
    auto_adjust=False
)

amzn.head()
```

```
[*****100%*****] 1 of 1 completed
```

```
{
  "summary": {
    "name": "amzn",
    "rows": 1255,
    "fields": [
      {
        "column": "Date",
        "dtype": "date",
        "min": "2021-07-19 00:00:00",
        "max": "2026-07-17 00:00:00",
        "num_unique_values": 1255,
        "samples": [
          "2026-04-23 00:00:00",
          "2021-12-09 00:00:00",
          "2021-09-29 00:00:00"
        ],
        "semantic_type": "",
        "description": ""
      },
      {
        "column": "Adj Close",
        "dtype": "number",
        "std": 46.77244065293703,
        "min":

```



```

## TYPE YOUR CODE BELOW
# Question 2: Download adjusted closing prices
# for the last 60 months

all_stocks_list = ["AMZN", "META", "IBM", "MMM"]

all_stocks = yf.download(
    all_stocks_list,
    start=start,
    auto_adjust=False,
    progress=False
)

# Select Adjusted Close only
adjusted_close = all_stocks["Adj Close"].copy()

# Rename META to FB to match the assignment
adjusted_close = adjusted_close.rename(
    columns={"META": "FB"}
)

# Arrange the required column order
adjusted_close = adjusted_close[
    ["FB", "MMM", "IBM", "AMZN"]
]

adjusted_close.head()

{"summary":{"\n  \"name\": \"adjusted_close\", \n  \"rows\": 1255, \n
\n  \"fields\": [\n    {\n      \"column\": \"Date\", \n
\n  \"properties\": {\n    \"dtype\": \"date\", \n    \"min\":
\n  \"2021-07-19 00:00:00\", \n    \"max\": \"2026-07-17 00:00:00\", \n
\n  \"num_unique_values\": 1255, \n    \"samples\": [\n
\n  \"2026-04-23 00:00:00\", \n    \"2021-12-09 00:00:00\", \n
\n  \"2021-09-29 00:00:00\", \n    ], \n    \"semantic_type\": \"\", \n
\n  \"description\": \"\", \n    }, \n    {\n
\n  \"column\": \"FB\", \n    \"properties\": {\n      \"dtype\":
\n  \"number\", \n    \"std\": 197.30299191607162, \n    \"min\":
\n  88.13603210449219, \n    \"max\": 787.4192504882812, \n
\n  \"num_unique_values\": 1243, \n    \"samples\": [\n
\n  653.2411499023438, \n    773.833740234375, \n
\n  493.5634460449219, \n    ], \n    \"semantic_type\": \"\", \n
\n  \"description\": \"\", \n    }, \n    {\n      \"column\":
\n  \"MMM\", \n    \"properties\": {\n      \"dtype\": \"number\", \n
\n  \"std\": 30.059655654864976, \n    \"min\": 66.1995620727539, \n
\n  \"max\": 172.9356231689453, \n    \"num_unique_values\": 1225, \n
\n  \"samples\": [\n    86.16474151611328, \n
\n  97.76961517333984, \n    134.73736572265625, \n    ], \n
\n  \"semantic_type\": \"\", \n    \"description\": \"\", \n
\n  }, \n    {\n      \"column\": \"IBM\", \n    \"properties\": {\n

```

```

\"dtype\": \"number\",\\n          \"std\": 64.82655114727017,\\n
\"min\": 97.615478515625,\\n          \"max\": 329.2300109863281,\\n
\"num_unique_values\": 1245,\\n          \"samples\": [\\n
163.68052673339844,\\n          113.70966339111328,\\n
300.2852478027344\\n          ],\\n          \"semantic_type\": \"\",\\n
\"description\": \"\"\\n          }\\n          },\\n          {\\n          \"column\":
\"AMZN\",\\n          \"properties\": {\\n          \"dtype\": \"number\",\\n
\"std\": 46.77244065293703,\\n          \"min\": 81.81999969482422,\\n
\"max\": 274.989990234375,\\n          \"num_unique_values\": 1215,\\n
\"samples\": [\\n          183.1999969482422,\\n
186.64999389648438,\\n          173.12600708007812\\n          ],\\n
\"semantic_type\": \"\",\\n          \"description\": \"\"\\n          }\\n
n          }\\n          ]\\n          }\", \"type\": \"dataframe\", \"variable_name\": \"adjusted_close\"}

```

adjusted_close.tail()

```

{ \"summary\": { \\n  \"name\": \"adjusted_close\",\\n  \"rows\": 5,\\n
\"fields\": [\\n    {\\n      \"column\": \"Date\",\\n
\"properties\": {\\n        \"dtype\": \"date\",\\n        \"min\":
\"2026-07-13 00:00:00\",\\n        \"max\": \"2026-07-17 00:00:00\",\\n
\"num_unique_values\": 5,\\n        \"samples\": [\\n          \"2026-
07-14 00:00:00\",\\n          \"2026-07-17 00:00:00\",\\n
\"2026-07-15 00:00:00\"\\n          ],\\n        \"semantic_type\": \"\",\\n
n        \"description\": \"\"\\n        }\\n        },\\n        {\\n
\"column\": \"FB\",\\n        \"properties\": {\\n          \"dtype\":
\"number\",\\n          \"std\": 12.880941750550067,\\n          \"min\":
646.010009765625,\\n          \"max\": 681.3099975585938,\\n
\"num_unique_values\": 5,\\n          \"samples\": [\\n
661.0399780273438,\\n          646.010009765625,\\n
681.3099975585938\\n          ],\\n          \"semantic_type\": \"\",\\n
\"description\": \"\"\\n          }\\n          },\\n          {\\n          \"column\":
\"MMM\",\\n          \"properties\": {\\n          \"dtype\": \"number\",\\n
\"std\": 2.1132045508000474,\\n          \"min\": 156.5800018310547,\\n
\"max\": 161.77000427246094,\\n          \"num_unique_values\": 5,\\n
\"samples\": [\\n          156.5800018310547,\\n
159.83999633789062,\\n          160.52999877929688\\n          ],\\n
\"semantic_type\": \"\",\\n          \"description\": \"\"\\n          }\\n
n          },\\n          {\\n          \"column\": \"IBM\",\\n          \"properties\": {\\n
\"dtype\": \"number\",\\n          \"std\": 33.795342741033444,\\n
\"min\": 211.1999969482422,\\n          \"max\": 290.2300109863281,\\n
\"num_unique_values\": 5,\\n          \"samples\": [\\n
217.07000732421875,\\n          212.6699981689453,\\n
211.1999969482422\\n          ],\\n          \"semantic_type\": \"\",\\n
\"description\": \"\"\\n          }\\n          },\\n          {\\n          \"column\":
\"AMZN\",\\n          \"properties\": {\\n          \"dtype\": \"number\",\\n
\"std\": 3.3119481449065797,\\n          \"min\": 247.22999572753906,\\n
\"max\": 254.9600067138672,\\n          \"num_unique_values\": 5,\\n
\"samples\": [\\n          247.49000549316406,\\n
247.22999572753906,\\n          254.9600067138672\\n          ],\\n

```

```
\ "semantic_type\": \"\",\n      \ "description\": \"\",\n      }\n    }\n  ]\n}", "type": "dataframe"}
```

```
adjusted_close.shape
```

(1255, 4)

```
all_stocks.head()
```

```
{"type": "dataframe", "variable_name": "all_stocks"}
```

Question 3

Resample to Business Month-End

```
monthly_prices = adjusted_close.resample("BM").last()
```

```
monthly_prices.head()
```

```
/tmp/ipykernel_1988/693165147.py:4: FutureWarning: 'BM' is deprecated
and will be removed in a future version, please use 'BME' instead.
```

```
monthly_prices = adjusted_close.resample("BM").last()
```

```
{
  "summary": {
    "name": "monthly_prices",
    "rows": 61,
    "fields": [
      {
        "column": "Date",
        "dtype": "date",
        "min": "2021-07-30 00:00:00",
        "max": "2026-07-31 00:00:00",
        "num_unique_values": 61,
        "samples": [
          "2021-07-30 00:00:00",
          "2021-12-31 00:00:00",
          "2025-05-30 00:00:00"
        ],
        "semantic_type": "",
        "description": ""
      },
      {
        "column": "FB",
        "dtype": "number",
        "std": 198.98722664727774,
        "min": 92.34902954101562,
        "max": 770.913330078125,
        "num_unique_values": 61,
        "samples": [
          353.1983337402344,
          333.4220275878906,
          644.8784790039062
        ],
        "semantic_type": "",
        "description": ""
      },
      {
        "column": "MMM",
        "dtype": "number",
        "std": 30.67598713778284,
        "min": 70.33705139160156,
        "max": 170.40017700195312,
        "num_unique_values": 61,
        "samples": [
          138.467041015625,
          126.2268295288086,
          145.62594604492188
        ],
        "semantic_type": "",
        "description": ""
      },
      {
        "column": "IBM",
        "dtype": "number",
        "std": 66.28425954184686,
        "min": 98.70280456542969,
        "max": 304.5888366699219,
        "num_unique_values": 61,
        "samples": [
          110.81752014160156,
          112.66111755371094,
          252.59930419921875
        ],
        "semantic_type": "",
        "description": ""
      },
      {
        "column": "AMZN",
        "dtype": "number",
        "std": 110.81752014160156,
        "min": 112.66111755371094,
        "max": 252.59930419921875,
        "num_unique_values": 61,
        "samples": [
          110.81752014160156,
          112.66111755371094,
          252.59930419921875
        ],
        "semantic_type": "",
        "description": ""
      }
    ]
  }
}
```

```

\ "std\ ": 48.2056065617767,\n          \ "min\ ": 84.0,\n          \ "max\ ":
270.6400146484375,\n          \ "num_unique_values\ ": 60,\n
\ "samples\ ": [\n          166.37950134277344,\n
166.7169952392578,\n          186.97999572753906\n          ],\n
\ "semantic_type\ ": \ "\",\n          \ "description\ ": \ "\ "\n          }\n
n          }\n          ]\n          }","type":"dataframe","variable_name":"monthly_prices"}

```

```
monthly_prices.tail()
```

```

{"summary":{"\n  \ "name\ ": \ "monthly_prices\ ",\n  \ "rows\ ": 5,\n
\ "fields\ ": [\n    {\n      \ "column\ ": \ "Date\ ",\n
\ "properties\ ": {\n        \ "dtype\ ": \ "date\ ",\n        \ "min\ ":
\ "2026-03-31 00:00:00\ ",\n        \ "max\ ": \ "2026-07-31 00:00:00\ ",\n
\ "num_unique_values\ ": 5,\n        \ "samples\ ": [\n          \ "2026-
04-30 00:00:00\ ",\n          \ "2026-07-31 00:00:00\ ",\n
\ "2026-05-29 00:00:00\ "\n          ],\n        \ "semantic_type\ ": \ "\",\n
n        \ "description\ ": \ "\ "\n        }\n      },\n      {\n
\ "column\ ": \ "FB\ ",\n      \ "properties\ ": {\n        \ "dtype\ ":
\ "number\ ",\n        \ "std\ ": 36.40779797164005,\n        \ "min\ ":
563.2899780273438,\n        \ "max\ ": 646.010009765625,\n
\ "num_unique_values\ ": 5,\n        \ "samples\ ": [\n
611.3433837890625,\n        646.010009765625,\n
631.92431640625\n        ],\n        \ "semantic_type\ ": \ "\",\n
\ "description\ ": \ "\ "\n        }\n      },\n      {\n        \ "column\ ":
\ "MMM\ ",\n        \ "properties\ ": {\n          \ "dtype\ ": \ "number\ ",\n
\ "std\ ": 7.922422712352939,\n          \ "min\ ": 144.48277282714844,\n
\ "max\ ": 161.91000366210938,\n          \ "num_unique_values\ ": 5,\n
\ "samples\ ": [\n          145.76614379882812,\n
159.83999633789062,\n          153.1300048828125\n          ],\n
\ "semantic_type\ ": \ "\",\n          \ "description\ ": \ "\ "\n          }\n
n        },\n        {\n          \ "column\ ": \ "IBM\ ",\n          \ "properties\ ": {\n
\ "dtype\ ": \ "number\ ",\n          \ "std\ ": 35.85514589476945,\n
\ "min\ ": 212.6699981689453,\n          \ "max\ ": 297.79998779296875,\n
\ "num_unique_values\ ": 5,\n          \ "samples\ ": [\n
229.2924041748047,\n          212.6699981689453,\n
297.79998779296875\n          ],\n          \ "semantic_type\ ": \ "\",\n
\ "description\ ": \ "\ "\n          }\n        },\n        {\n          \ "column\ ":
\ "AMZN\ ",\n          \ "properties\ ": {\n            \ "dtype\ ": \ "number\ ",\n
\ "std\ ": 24.76950056811034,\n            \ "min\ ": 208.27000427246094,\n
\ "max\ ": 270.6400146484375,\n            \ "num_unique_values\ ": 5,\n
\ "samples\ ": [\n            265.05999755859375,\n
247.22999572753906,\n            270.6400146484375\n            ],\n
\ "semantic_type\ ": \ "\",\n            \ "description\ ": \ "\ "\n            }\n
n          }\n          ]\n          }","type":"dataframe"}

```

```
monthly_prices.shape
```

```
(61, 4)
```

```
monthly_prices.isna().sum()
```



```
Ticker
FB      0
MMM     0
IBM     0
AMZN    0
dtype: int64
```

```
monthly_prices = monthly_prices.dropna()
```

```
# last business (trading) day of the month
```

```
monthly_prices.head()
```

```
{"summary":{"\n  \"name\": \"monthly_prices\",\n  \"rows\": 61,\n  \"fields\": [\n    {\n      \"column\": \"Date\",\n      \"properties\": {\n        \"dtype\": \"date\",\n        \"min\": \"2021-07-30 00:00:00\",\n        \"max\": \"2026-07-31 00:00:00\",\n        \"num_unique_values\": 61,\n        \"samples\": [\n          \"2021-07-30 00:00:00\",\n          \"2021-12-31 00:00:00\",\n          \"2025-05-30 00:00:00\"\n        ],\n        \"semantic_type\": \"\",\n        \"description\": \"\"\n      },\n      \"column\": \"FB\",\n      \"properties\": {\n        \"dtype\": \"number\",\n        \"std\": 198.98722664727774,\n        \"min\": 92.34902954101562,\n        \"max\": 770.913330078125,\n        \"num_unique_values\": 61,\n        \"samples\": [\n          353.1983337402344,\n          333.4220275878906,\n          644.8784790039062\n        ],\n        \"semantic_type\": \"\",\n        \"description\": \"\"\n      },\n      \"column\": \"MMM\",\n      \"properties\": {\n        \"dtype\": \"number\",\n        \"std\": 30.67598713778284,\n        \"min\": 70.33705139160156,\n        \"max\": 170.40017700195312,\n        \"num_unique_values\": 61,\n        \"samples\": [\n          138.467041015625,\n          126.2268295288086,\n          145.62594604492188\n        ],\n        \"semantic_type\": \"\",\n        \"description\": \"\"\n      },\n      \"column\": \"IBM\",\n      \"properties\": {\n        \"dtype\": \"number\",\n        \"std\": 66.28425954184686,\n        \"min\": 98.70280456542969,\n        \"max\": 304.5888366699219,\n        \"num_unique_values\": 61,\n        \"samples\": [\n          110.81752014160156,\n          112.66111755371094,\n          252.59930419921875\n        ],\n        \"semantic_type\": \"\",\n        \"description\": \"\"\n      },\n      \"column\": \"AMZN\",\n      \"properties\": {\n        \"dtype\": \"number\",\n        \"std\": 48.2056065617767,\n        \"min\": 84.0,\n        \"max\": 270.6400146484375,\n        \"num_unique_values\": 60,\n        \"samples\": [\n          166.37950134277344,\n          166.7169952392578,\n          186.97999572753906\n        ],\n        \"semantic_type\": \"\",\n        \"description\": \"\"\n      }\n    ]\n  ],\n  \"type\": \"dataframe\",\n  \"variable_name\": \"monthly_prices\"}
```



```

from pandas.plotting import autocorrelation_plot
import matplotlib.pyplot as plt

# Question 4
# Autocorrelation of Adjusted Month-End Close Prices

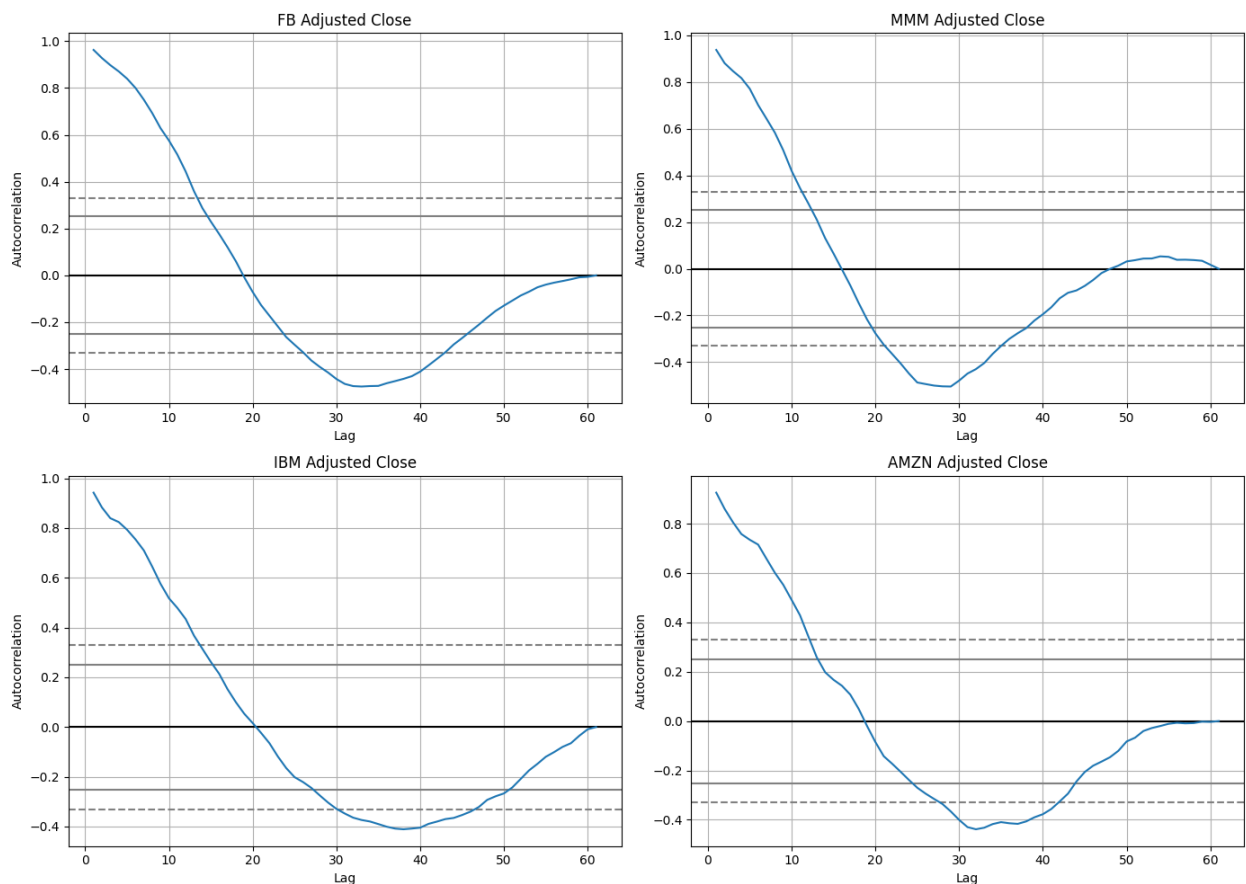
fig, axes = plt.subplots(2, 2, figsize=(14, 10))

stocks = monthly_prices.columns

for ax, stock in zip(axes.flatten(), stocks):
    autocorrelation_plot(monthly_prices[stock], ax=ax)
    ax.set_title(f"{stock} Adjusted Close")
    ax.set_xlabel("Lag")
    ax.set_ylabel("Autocorrelation")

plt.tight_layout()
plt.show()

```



```

print("Lag-1 Autocorrelation")

for stock in monthly_prices.columns:
    print(

```

```
f"{stock}:",
round(monthly_prices[stock].autocorr(lag=1), 4)
)
```

Lag-1 Autocorrelation

FB: 0.9741

MMM: 0.9584

IBM: 0.9528

AMZN: 0.9453

Are the stock prices autocorrelated?

Yes.

The adjusted month-end closing prices show strong positive autocorrelation. This means that the stock price in one month is highly related to the price in the previous month. Stock prices generally move gradually over time rather than changing independently from one month to the next. Therefore, the autocorrelation plots display strong positive correlations at lower lags, indicating persistence in the price series.

Question 5

Calculate Monthly Returns using the Shift Trick

```
monthly_returns = (
    monthly_prices - monthly_prices.shift(1)
) / monthly_prices.shift(1)
```

Remove the first row (contains NaN)

```
monthly_returns = monthly_returns.dropna()
```

```
monthly_returns.head()
```

```
{"summary": "{\n  \"name\": \"monthly_returns\",\n  \"rows\": 60,\n  \"fields\": [\n    {\n      \"column\": \"Date\",\n      \"properties\": {\n        \"dtype\": \"date\",\n        \"min\": \"2021-08-31 00:00:00\",\n        \"max\": \"2026-07-31 00:00:00\",\n        \"num_unique_values\": 60,\n        \"samples\": [\n          \"2021-08-31 00:00:00\",\n          \"2022-01-31 00:00:00\",\n          \"2024-08-30 00:00:00\"\n        ],\n        \"semantic_type\": \"\",\n        \"description\": \"\"\n      }\n    },\n    {\n      \"column\": \"FB\",\n      \"properties\": {\n        \"dtype\": \"number\",\n        \"std\": 0.12148894070942562,\n        \"min\": -0.326342406948737,\n        \"max\": 0.2677113690821796,\n        \"num_unique_values\": 60,\n        \"samples\": [\n          0.06477697567473155,\n          -0.06864866492585789,\n          0.09788748068646203\n        ],\n        \"semantic_type\": \"\",\n        \"description\": \"\"\n      }\n    },\n    {\n      \"column\": \"MMM\",\n      \"properties\": {\n        \"dtype\": \"number\",\n        \"std\": 0.0816400534755541,\n        \"min\": -0.1369374956133298,\n        \"max\": 0.2481652220814882,\n        \"num_unique_values\": 60,\n        \"samples\": [\n          0.0816400534755541,\n          -0.1369374956133298,\n          0.2481652220814882\n        ]\n      }\n    }\n  ]\n}
```

```

{"samples\": [\n          -0.00865649260086642,\n          0.06536034306361943,\n          0.061670621235255886,\n          ],\n \"semantic_type\": \"\",\n \"description\": \"\",\n \"column\": \"IBM\",\n \"properties\": {\n \"dtype\": \"number\",\n \"std\": 0.09151914042037036,\n \"min\": -0.24373242547849416,\n \"max\": 0.29877825157232957,\n \"num_unique_values\": 60,\n \"samples\": [\n 0.007064129242990431,\n -0.0006733385170624915,\n 0.06119433957498563,\n ],\n \"semantic_type\": \"\",\n \"description\": \"\",\n \"column\":\n \"AMZN\",\n \"properties\": {\n \"dtype\": \"number\",\n \"std\": 0.09659526013439333,\n \"min\": -0.23752509313621048,\n \"max\": 0.2726748553374953,\n \"num_unique_values\":\n 60,\n \"samples\": [\n 0.043034169499993905,\n -0.10282991021845668,\n -0.04535242229813625,\n ],\n \"semantic_type\": \"\",\n \"description\": \"\"\n }\n }],\n \"type\": \"dataframe\", \"variable_name\": \"monthly_returns\"}

```

```
monthly_returns.isna().sum()
```

```

Ticker
FB      0
MMM     0
IBM     0
AMZN    0
dtype: int64

```

```

from pandas.plotting import autocorrelation_plot
import matplotlib.pyplot as plt

```

```
fig, axes = plt.subplots(2, 2, figsize=(14, 10))
```

```
stocks = monthly_returns.columns
```

```

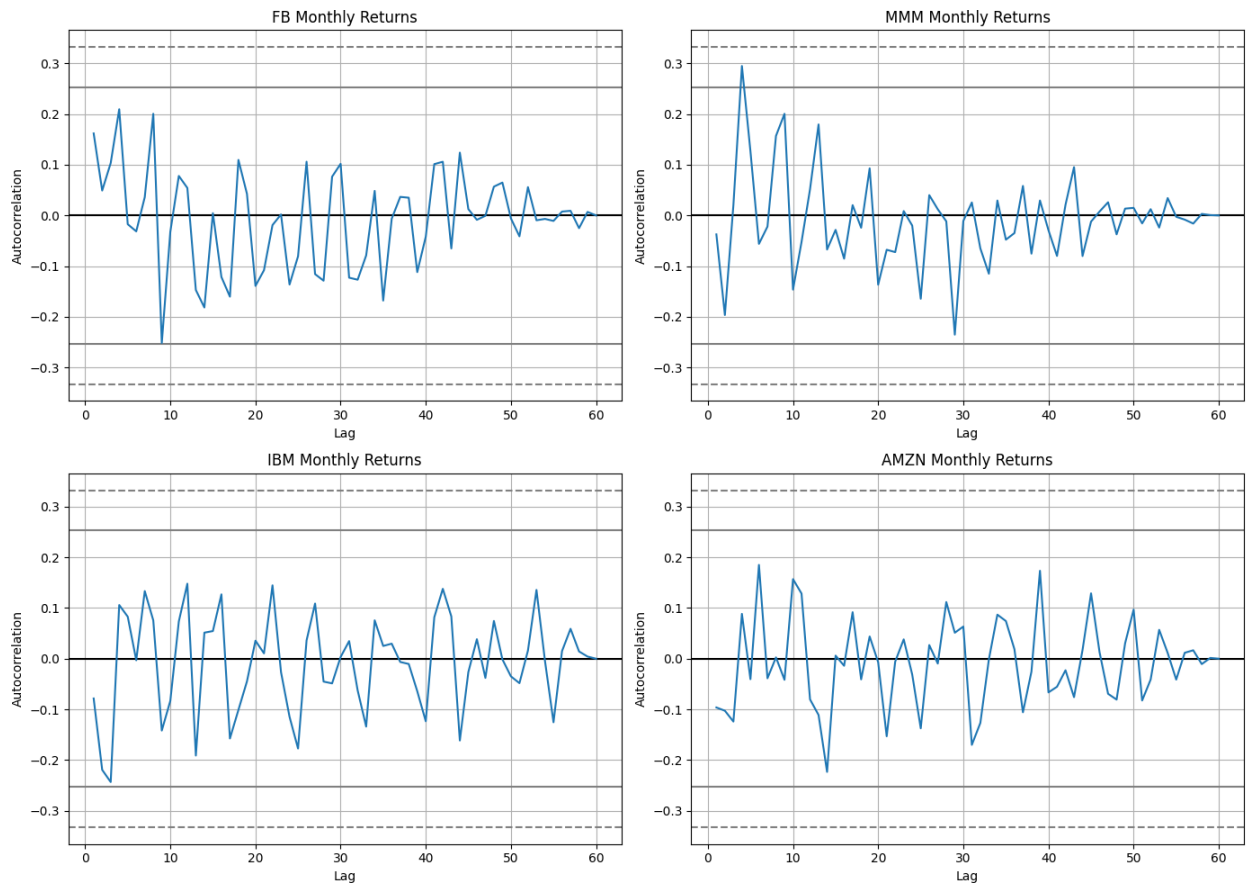
for ax, stock in zip(axes.flatten(), stocks):
    autocorrelation_plot(monthly_returns[stock], ax=ax)
    ax.set_title(f"{stock} Monthly Returns")
    ax.set_xlabel("Lag")
    ax.set_ylabel("Autocorrelation")

```

```

plt.tight_layout()
plt.show()

```



```
print("Lag-1 Autocorrelation of Monthly Returns")

for stock in monthly_returns.columns:
    print(
        f"{stock}:",
        round(monthly_returns[stock].autocorr(lag=1), 4)
    )
```

Lag-1 Autocorrelation of Monthly Returns

FB: 0.1636

MMM: -0.0374

IBM: -0.0846

AMZN: -0.0964

Are the monthly returns autocorrelated?

Generally, no.

The monthly returns exhibit weak or no significant autocorrelation. Most autocorrelation values are close to zero, indicating that returns from one month do not consistently predict returns in the following month. Unlike stock prices, which usually move gradually over time, monthly returns fluctuate around zero and are more random. This behavior is consistent with the efficient

market hypothesis, which suggests that past returns provide limited information for predicting future returns.

```
# Question 6
# Combine all four return series

returns_df = monthly_returns[
    ["FB", "MMM", "IBM", "AMZN"]
].copy()

# Remove any missing values
returns_df = returns_df.dropna()

returns_df.head()

{"summary":{"name": "returns_df", "rows": 60, "fields": [{"column": "Date", "properties": {"dtype": "date", "min": "2021-08-31 00:00:00", "max": "2026-07-31 00:00:00", "num_unique_values": 60, "samples": ["2021-08-31 00:00:00", "2022-01-31 00:00:00", "2024-08-30 00:00:00"], "semantic_type": "", "description": ""}], [{"column": "FB", "properties": {"dtype": "number", "std": 0.12148894070942562, "min": -0.326342406948737, "max": 0.2677113690821796, "num_unique_values": 60, "samples": [0.06477697567473155, -0.06864866492585789, 0.09788748068646203], "semantic_type": "", "description": ""}], [{"column": "MMM", "properties": {"dtype": "number", "std": 0.0816400534755541, "min": -0.1369374956133298, "max": 0.2481652220814882, "num_unique_values": 60, "samples": [-0.00865649260086642, 0.06536034306361943, 0.061670621235255886], "semantic_type": "", "description": ""}], [{"column": "IBM", "properties": {"dtype": "number", "std": 0.09151914042037036, "min": -0.24373242547849416, "max": 0.29877825157232957, "num_unique_values": 60, "samples": [0.007064129242990431, -0.0006733385170624915, 0.06119433957498563], "semantic_type": "", "description": ""}], [{"column": "AMZN", "properties": {"dtype": "number", "std": 0.09659526013439333, "min": -0.23752509313621048, "max": 0.2726748553374953, "num_unique_values": 60, "samples": [0.043034169499993905, -0.10282991021845668, -0.04535242229813625]}]}
```

```
\["semantic_type\": \["\", \n      \["description\": \["\"\\n      }\n    ]\n  ]\n]", "type": "dataframe", "variable_name": "returns_df"]
```

```
returns_df.isna().sum()
```

Ticker

FB 0

MMM 0

IBM 0

AMZN 0

```
dtype: int64
```

```
correlation_matrix = returns_df.corr()
```

```
print("Correlation Matrix")
```

correlation_matrix

Correlation Matrix

```
{
  "summary": {
    "name": "correlation_matrix",
    "rows": 4,
    "fields": [
      {
        "column": "Ticker",
        "properties": {
          "dtype": "string",
          "num_unique_values": 4,
          "samples": [
            "AMZN",
            "FB",
            "IBM",
            "MMM"
          ],
          "semantic_type": "",
          "description": ""
        },
        "column": "FB",
        "properties": {
          "dtype": "number",
          "std": 0.41499308935493384,
          "min": 0.09191793724207278,
          "max": 1.0,
          "num_unique_values": 4,
          "samples": [
            0.09191793724207278,
            0.4638349829933582,
            1.0
          ],
          "semantic_type": "",
          "description": ""
        },
        "column": "MMM",
        "properties": {
          "dtype": "number",
          "std": 0.41354341595143507,
          "min": 0.09191793724207278,
          "max": 1.0,
          "num_unique_values": 4,
          "samples": [
            1.0,
            0.18176605773429308,
            0.09191793724207278
          ],
          "semantic_type": "",
          "description": ""
        },
        "column": "IBM",
        "properties": {
          "dtype": "number",
          "std": 0.4221409076069167,
          "min": 0.07665893492066866,
          "max": 1.0,
          "num_unique_values": 4,
          "samples": [
            0.29637585812457673,
            0.07665893492066866,
            0.15370068400351108
          ],
          "semantic_type": "",
          "description": ""
        },
        "column": "AMZN",
        "properties": {
          "dtype": "number",
          "std": 0.41332534705961305,
          "min": 0.07665893492066866,
          "max": 1.0,
          "num_unique_values": 4,
          "samples": [
            0.18176605773429308,
            1.0,
            0.4638349829933582
          ],
          "semantic_type": ""
        }
      ]
    }
  }
}
```

```
\n"description\\": \\\"\\\"\\n      }\\n      }\\n  ]\\n}\\n","type":"dataframe","variable_name":"correlation_matrix"}

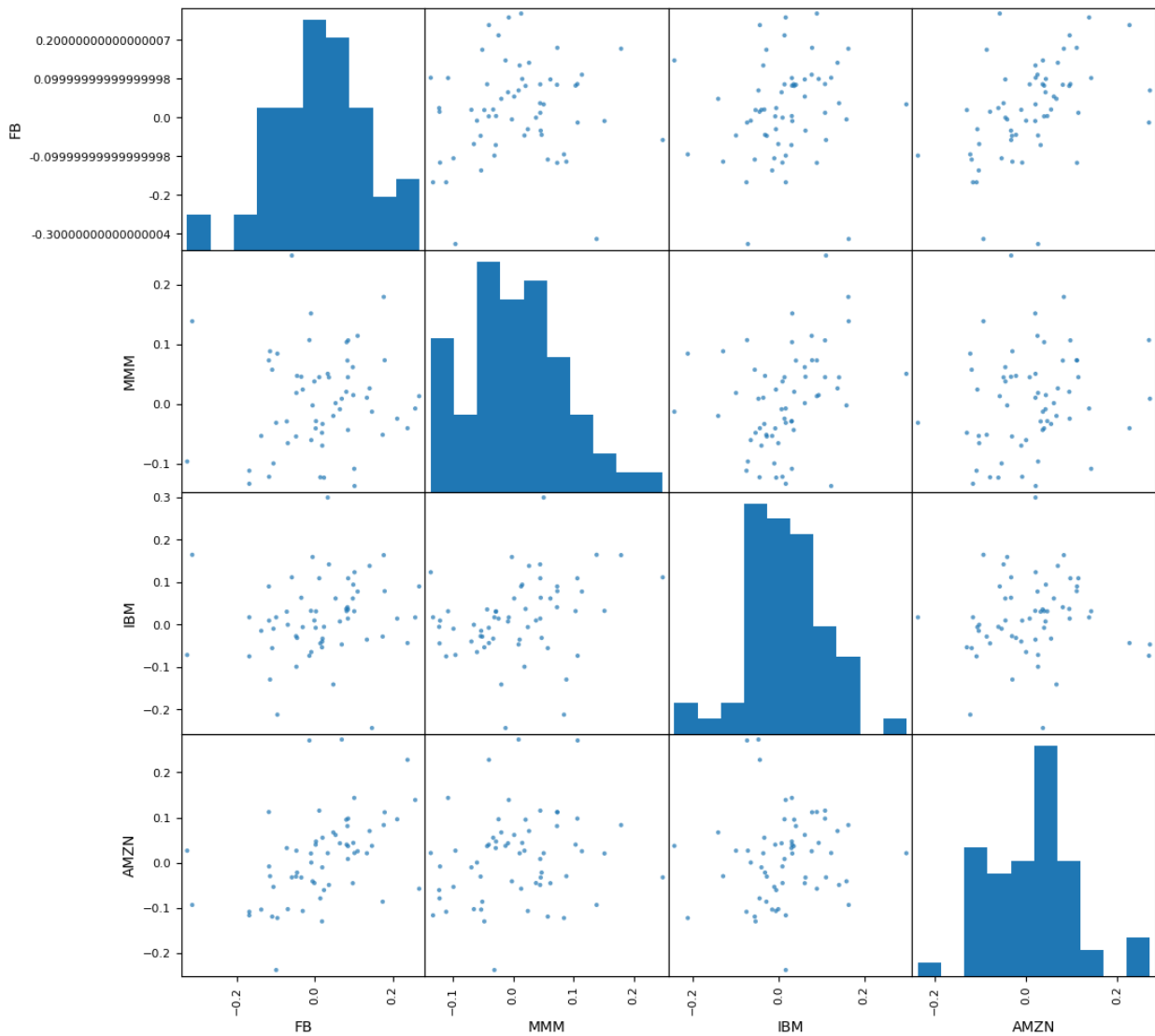
from pandas.plotting import scatter_matrix
import matplotlib.pyplot as plt

scatter_matrix(
    returns_df,
    figsize=(12, 12),
    diagonal="hist",
    alpha=0.7
)

plt.suptitle(
    "Scatter Matrix of Monthly Stock Returns",
    fontsize=16
)

plt.show()
```


Scatter Matrix of Monthly Stock Returns



Is there correlation?

Yes, there is some positive correlation, but it is generally moderate rather than perfect.

The scatter plot matrix and correlation matrix indicate that several stock pairs have positive correlations, meaning they tend to move in the same direction during some months. However, the relationships are not perfectly linear, as shown by the scattered point patterns. Technology companies such as Facebook (Meta) and Amazon may exhibit stronger correlations because they operate in related sectors, while 3M and IBM may show weaker relationships due to differences in their industries. Overall, the correlations suggest that market-wide economic factors influence all stocks, but each company also experiences unique business-specific risks.

```
print("Missing Values")
print(returns_df.isna().sum())
```

```
print("\nShape")
print(returns_df.shape)
```

```
print("\nColumns")
print(returns_df.columns)
```

Missing Values

Ticker

FB 0

MMM 0

IBM 0

AMZN 0

dtype: int64

Shape

(60, 4)

Columns

Index(['FB', 'MMM', 'IBM', 'AMZN'], dtype='object', name='Ticker')